2. QuickStart Guide to Tuning PostgreSQL

PostgreSQL often gets accused of being rather slow. In earlier versions, back in the "dark ages" of 7.0 or 6.5, that was certainly true. In modern days, this is typically the result of taking an install with all of the defaults from what is generated by the source distribution, perhaps combined with running an application already specifically tuned for optimal behaviour with some other database.

The defaults are configured to be able to function on all platforms on which **PostgreSQL** runs, and are definitely *not* optimal for typical usage.

Fortunately, it is quite straightforward to achieve *massive* improvements by changing a very few things. Here is a sort of " top ten" list of things that should be done to improve performance, and why:

- Increase, in `postgresql.conf`, the value of `shared_buffers`.

  The defaults are low because on some platforms (Solaris and SGI, notably) having large values requires invasive action like recompiling the kernel.

  The shared buffers are used to hold query results that are in progress. The default of 512K is quite likely to be low. The "rule of thumb" is to estimate the " right value " to be between 5% and 15% of total system memory. 15% would be on the high side, where the server will exclusively be doing database work. A lower number would be appropriate if the server is hosting other large applications.

  Note that this value does not include any OS file caching that may take place; look below at `effective_cache_size`, which addresses that. Note that any time records are updated, they must be pushed out to disk, thereby invalidating *both* this cache as well as the operating system's own filesystem cache. That is why this buffer size shouldn't be made too large.

  If the value you try exceeds the amount of shared memory supported by your operating system kernel, you will get a message similar to:

  ```
  IpcMemoryCreate: shmget(key=5432001, size=415776768, 03600) failed:
  Invalid argument

  This error usually means that PostgreSQL's request for a shared memory
  segment exceeded your kernel's SHMMAX parameter.  You can either
  reduce the request size or reconfigure the kernel with larger SHMMAX.
  To reduce the request size (currently 415776768 bytes), reduce
  PostgreSQL's shared_buffers parameter (currently 50000) and/or
  its max_connections parameter (currently 12).
  ```

  - Remedying this may be accomplished, on Linux, by the following command:

    ```
    echo VAL > /proc/sys/kernel/shmmax
    ```

- On **FreeBSD**, you should be able to accomplish the equivalent, by modifying the `/etc/sysctl.conf` file, modifying `kern.ipc.shmmax`=VAL.
- On Solaris, it is necessary to recompile the kernel to accomplish this; procedures for other operating systems may vary, but likely resemble one of the procedures described here.
- On some systems, the following may work:

  ```
  sysctl -w sys.kernel.shmmax=VAL
  ```

- Increase, in `postgresql.conf`, the value of `effective_cache_size`.

  This parameter indicates to the query optimizer how much OS filesystem cache you expect to have available. On a server used solely for running **PostgreSQL**, this might represent most of your system's physical memory.

- Run `VACUUM` on database tables frequently.

  Any time records are deleted or updated in a table, this leads to generating " dead tuples" for this table. They are *not* automatically cleaned out; that requires running `VACUUM` . If the table is *not* vacuumed, the postmaster will have to rummage through the dead data, only to discard it as useless. It may be no big deal if there are 10,000 dead tuples in a table with a million entries, but performance will be *bad* if (say) a table of financial balances has a few hundred live entries combined with hundreds of thousands of obsolete dead tuples.

  At one time, `VACUUM` locked whatever table was being cleaned; that has long changed, so that it now does not block activity on the system. Seemingly paradoxically, the more often you run `VACUUM` , the *faster* it tends to run, and the more likely it is that it will do some good.

  If there get to be *really a lot* of dead tuples in a table, there may be more free space than the *free space map* knows to manage, and a proper cleanup may require the other version, called `VACUUM FULL` . You can get a feel for the usage of the free space map when running `VACUUM VERBOSE`; the last two lines summarize FSM usage.

  A recently developed program, **pg_autovacuum**, monitors database updates, and automatically initiates `VACUUM`s on just those tables that need cleaning.

  There are system tables that are updated heavily under varying conditions; if they are updated heavily, they must be vacuumed too, otherwise performance will suffer:

  - When temporary tables are created and dropped, this creates entries in `pg_class` and `pg_attribute`.
  - When your applications publish events using `NOTIFY`, entries are created in `pg_listener`
  - Every time you `ANALYZE` tables, this replaces statistical entries in `pg_statistic`.

- Run `ANALYZE` on the database periodically.

  PostgreSQL's query optimizer is a cost-based system, and therefore needs to have some statistics on what data the various tables on the system contain.

  Until you run `ANALYZE`, the system may have no better ideas than to do sequential scans of all the tables involved in queries.

  The same is true for other cost-based optimizers; I have also seen **Oracle** bog down horribly on queries until some statistics built up at which point those queries sped up immensely.

- *Don't* run `VACUUM FULL` too often.

  While it is the only way of actually reclaiming disk space back to the operating system, and sometimes needs to be done if a table or index has gotten exceedingly fragmented across pages, it has three demerits:

  - Running `VACUUM FULL` is expensive, as it has to read through all the pages of the table;
  - It effectively puts a table lock on the table for the duration of the operation, blocking other use of the table.
  - If you are likely to insert new data into the table that will fill the empty space back in, the work will have been all for nothing.

  If you are running `VACUUM FULL` a lot, then there is probably a problem with the way you are using the database.

- Move the WAL files to a separate physical disk.

  That is, set the directory `pg_xlog` to point to a directory on a separate disk from that which the data is stored on. These files, that contain the most recent updates to the database, are *heavily* updated, and if you can separate these updates from those concurrently taking place on database tables, you can get an *easy* performance boost, often on the order of 25%.

  It may also be helpful to ensure log files are being written to a separate physical device.

  If your requirements are more sophisticated, it may even be worthwhile to move individual database files to separate disks if you have enough disk drives to do so.

- Increase, in `postgresql.conf`, the value of `sort_mem`

  If you do a lot of complex sorts, and have a lot of memory, then increasing this parameter allows **PostgreSQL** to do larger in-memory sorts which, unsurprisingly, will be faster than disk-based equivalents.

  Mind you, this size is applied to *each and every sort* done by each user, and complex queries can use multiple sort buffers. Set it to 50MB, and have 30 users submitting queries, and you are soon using 1.5GB of real memory. Furthermore, if a query involves doing merge sorts of 8 tables, that requires 8 times `sort_mem`.

  This is where " Data Warehouse" systems, where users are submitting *very large queries*, can readily make use of multiple gigabytes of memory. 32 bit architectures, where there is a hard limit of 2GB per process, are starting to look pretty creaky. As AMD x86-64 systems make it inexpensive to deploy systems with tens of gigabytes of memory, that becomes quite a big deal...

- Reduce, in `postgresql.conf`, the value of `random_page_cost`

  If you have particularly fast disks, as commonly found with RAID arrays of SCSI disks, it may be appropriate to lower this figure, which will encourage the query optimizer to use random access index scans.

- Increase, in `postgresql.conf`, the value of `max_fsm_pages` and `max_fsm_relations`

  The Free Space Map is used to track where there are dead tuples that may be reclaimed. You will only get effective nonblocking `VACUUM` queries if the dead tuples can be listed in the Free Space Map.

As a result, if you do not plan to run `VACUUM` frequently, and if you expect a lot of updates, you should ensure these values are usefully large. It should be easy enough to set `max_fsm_relations` high enough; the problem that will more typically occur is when `max_fsm_pages` is not set high enough. Once the Free Space Map is full, `VACUUM` will be unable to track further dead pages. In a busy database, this needs to be set much higher than 1000...

This represents a set of actions that are *highly* specific to **PostgreSQL** ; if you go through these activities, you should see substantial improvements in performance over the " default" settings.

After that, it also makes sense to look to additional techniques to tune specific queries. Typically, this involves examining the query plans for particular queries, and experimenting to see what changes improve those plans. This commonly involves adding indices that allow your queries to be efficiently selective about what records they retrieve. Evaluating the efficiency of queries is a bit of a "black art" where it simply takes time and practice to recognize the goodness of a query plan.

Google Search

Ads by Google ► PostgreSQL ► SQL Tuning ► SQL Queries ► What Linu

If this was useful, let others know by an Affero rating

Contact me at cbbrowne@acm.org